AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
INTERNATIONAL CREDIT HOURS ENGINEERING PROGRAMS (ICHEP)

Hotel Management System

Milestone 2

CSE241
"Object-Oriented Programming"

Submitted to:

Dr. Mahmoud Ibrahim Khalil
Eng. Mazen Tarek Samy

Submitted by:

Names	IDs
Abdallah Nageh Salama	25P0418
Awsam Nady Shehata	25P0419
Beshoy Hany Malak	25P0441
Hassan Abdalla Ahmed	25P0302
Sameh Amged Makram	25P0429

Links:

Github: https://github.com/abdallahNageh/Desktop-Hotel-Reservation-System/blob/main/final%20oop%20project.rar

Demo Video: https://youtu.be/0opY47Rrk0Y?feature=shared

Executive Summary

This report documents Milestone 2 of the Hotel Management System, developed for the CSE241 Object-Oriented Programming course at Ain Shams University. Building directly on the solid backend established in Milestone 1, this phase introduces a fully interactive JavaFX graphical user interface, multi-threading for background operations, and a real-time client-server chat system implemented with Java Sockets.

The primary objectives achieved in this milestone include:

- GUI Implementation: Designing and implementing a complete multi-screen JavaFX application covering all user roles (Admin, Receptionist, and Guest), with FXML-based layouts, CSS styling, and role-based navigation.
- Multi-threading: Using background Java threads and Platform.runLater() to keep the UI responsive during data loading and network operations.
- Networking: Building a multi-client TCP chat server (ChatServer) and a corresponding client (ChatClient) that allow guests and receptionists to exchange real-time messages.
- Integration: Seamlessly connecting all UI controllers to the existing backend domain model from Milestone 1 without modifying core business logic.

Design Decisions & System Architecture

MVC-Inspired Layered Architecture

We maintained a strict separation between the backend domain model (BackEnd package) and the presentation layer (com.example.hotel_project package). Every JavaFX controller communicates with the domain exclusively through HotelDatabase static methods and domain class methods — no UI logic leaks into backend classes. This makes the codebase maintainable and easily testable.

FXML + CSS Separation of Concerns

All screen layouts are defined in FXML files, keeping UI structure completely separate from Java controller logic. CSS stylesheets are applied per-dashboard to give each role a consistent visual identity. This approach also makes it trivial to redesign any screen without touching controller code.

Scene Switching Strategy

Navigation between screens uses a Stage-level scene replacement pattern: each controller holds a reference to the current Stage, loads the target FXML, and calls stage.setScene(). The Receptionist hub screen (ReceptionistController) uses an embedded StackPane content area instead, swapping child panes without replacing the whole scene — this keeps the sidebar navigation persistent.

Thread Safety & Platform.runLater()

The chat connection and all incoming message callbacks run on background daemon threads. Every UI update triggered from those threads is wrapped in Platform.runLater() to post the update onto the JavaFX Application Thread, strictly following JavaFX's single-threaded UI model and preventing concurrent modification exceptions.

Functional Interface for Chat Callbacks

ChatClient exposes a @FunctionalInterface (MessageListener) for incoming message callbacks. Controllers pass a lambda (this::onMessageReceived), completely decoupling the networking layer from the UI. This is a clean application of the Observer pattern using Java 8+ functional programming.

JavaFX GUI — Screens & UI Components

The application contains 16 FXML screens organized by user role. All screens use proper JavaFX layouts (BorderPane, VBox, HBox, GridPane, StackPane) and display appropriate feedback messages using Alert dialogs, Label status messages, and row color-coding.

Screen (FXML)	Controller	Role	Key Components
login.fxml	loginControl	All	TextField, PasswordField, ToggleGroup (role selector), error Label
Register.fxml	RegisterController	Guest	TextField, PasswordField, DatePicker, ComboBox (gender), validation Label
adminDashboard.fxml	adminDashboardControl	Admin	Welcome Label, navigation Buttons to sub-modules, logout
RoomManagement.fxml	ManageRoomsController	Admin	TableView<Room>, TextField inputs, ComboBox<RoomType>, MenuButton (multi-select amenities)
RoomType.fxml	RoomTypeController	Admin	TableView<RoomType>, ComboBox<RoomTypeName>, TextField, CRUD Buttons
amenityboard.fxml	amenityController	Admin	TableView<Amenity>, TextField (name/desc), Add/Edit/Delete Buttons
GuestDashBoard.fxml	GuestController	Guest	Welcome/balance Label, navigation Buttons

Screen (FXML)	Controller	Role	Key Components
			(rooms, reservations, chat, logout)
ViewAvailableRoom.fxml	ViewAvailableRoomController	Guest	TreeView<String> (hierarchical room display), DatePicker x2, Reserve Button
MyReservations.fxml	MyReservationControl	Guest	TableView<Reservation> with color-coded rows, Cancel/Checkout Buttons, balance Label
ManageReceptionistWindow.fxml	ReceptionistController	Receptionist	Sidebar navigation + StackPane content area (hub screen)
ManageReceptionistDashboard.fxml	ReceptionistDashboardController	Receptionist	Summary Labels: total guests, available rooms, active reservations
ManageReceptionistCheckIn.fxml	ReceptionistCheckInController	Receptionist	TextField (guest username), ComboBox<PaymentMethod>, invoice VBox (hidden until checkout)
ManageReceptionistReservations.fxml	ReceptionistReservationsController	Receptionist	TableView<Reservation> with color rows, Confirm Button, status Label
ManageReceptionistGuests.fxml	ReceptionistGuestsController	Receptionist	TableView<Guest> showing username, DOB, gender, address, balance
ManageReceptionistRooms.fxml	ReceptionistRoomsController	Receptionist	TableView<Room> showing number, type, price, availability
ChatWindow.fxml	ChatWindowController	Guest & Receptionist	TextArea (messages), TextField (input), ComboBox (recipients), Connect/Send Buttons, status Label

UI Feedback & Validation

Every user action produces clear feedback. Form validation (e.g., empty fields, invalid numbers, mismatched passwords, age restrictions) is checked before any backend call and displayed via inline error Labels. Successful operations trigger Alert.AlertType.INFORMATION dialogs. Destructive operations (cancel reservation) use a CONFIRMATION dialog before proceeding. The MyReservations and ReceptionistReservations tables use row color-coding (yellow = PENDING, blue = CONFIRMED, green = CHECKED_IN/COMPLETED, red = CANCELLED) for instant visual status feedback.

Multi-threading Implementation

Chat Connection Thread

Connecting to the TCP chat server is a blocking I/O operation. ChatWindowController spawns a dedicated daemon Thread for the connection attempt so the JavaFX UI never freezes. Once connected, the button text and color update via Platform.runLater(), and a background listener thread inside ChatClient continuously reads incoming messages from the server's InputStream.

ChatClient Listener Thread

Inside ChatClient.startListening(), a daemon thread blocks on BufferedReader.readLine() in a loop. When a message arrives, it is split on ':' and passed to the MessageListener callback. The controller's lambda then wraps the UI update in Platform.runLater() ensuring thread safety. The thread is set as daemon so it does not prevent JVM shutdown when the application closes.

ChatServer Client Handlers

ChatServer accepts new connections in a loop on the main server thread. For every accepted Socket, a new ClientHandler (Runnable inner class) is created and launched in its own Thread. This gives the server true concurrent multi-client support. A synchronized HashSet<ClientHandler> tracks all active connections; the synchronized keyword on broadcastMessage() and sendMessageTo() prevents race conditions when multiple clients send simultaneously.

Thread	Location	Purpose	Thread-Safety Mechanism
Connection Thread	ChatWindowController.connectToServer()	Non-blocking server connect	Platform.runLater() for UI updates
Listener Thread	ChatClient.startListening()	Continuous incoming message read	Platform.runLater() in MessageListener callback
ClientHandler Thread (×N)	ChatServer — one per client	Per-client message processing	synchronized methods on ChatServer
Server Accept Loop	ChatServer.start()	Accept new TCP connections	volatile boolean running flag for clean shutdown

Networking — Real-Time Chat System

Architecture

The chat system follows a classic client-server model. A single ChatServer instance listens on TCP port 5555. Every user who opens the chat window creates a ChatClient that connects to localhost:5555. The server acts as a message broker: it routes direct messages between named recipients and can broadcast to all connected clients.

Message Protocol

Messages follow a simple text-based protocol over UTF-8 encoded streams:

Direction	Format	Meaning
Client → Server	LOGIN:username:role	Registers the client's identity on the server after connecting.
Client → Server	TO:recipient:message	Sends a private message to a specific named user.
Client → Server	BROADCAST:message	Sends a message to every connected client (receptionist announcement).
Server → Client	sender:message	Delivers an incoming message, where sender is the originator's username.

Guest Chat Flow

A guest opens the Chat window, selects a receptionist from the ComboBox (populated from HotelDatabase.getReceptionists()), clicks Connect, then types and sends messages. The connection runs in a background thread so the UI stays responsive. Received messages are appended to the TextArea via Platform.runLater().

Receptionist Chat Flow

A receptionist's chat ComboBox shows all registered guests plus a 'Broadcast to All Guests' option. Selecting a guest sends a direct message; selecting broadcast calls chatClient.sendBroadcast() which routes through the server to every connected client. This allows receptionists to announce hotel-wide notifications in real-time.

Server Lifecycle

The ChatServer is started in MainClass.start() in a background daemon thread before the JavaFX UI is shown, ensuring it is always running. A JVM shutdown hook calls server.stop() for graceful socket cleanup. The server uses a volatile boolean running flag to signal the accept loop and all ClientHandler threads to exit cleanly.

Detailed Contributions by Team Members

Sameh Amged (Login Screen & Admin Dashboard)

- Login Screen (loginControl + login.fxml + login.css): Designed and implemented the main entry point of the application. Built the role-selector ToggleGroup (Admin / Receptionist / Guest), credential TextFields, and inline error Label. Wired up Admin.login(), Receptionist.login(), and Guest.login() and implemented role-based scene routing to the correct dashboard after successful authentication.
- Register Navigation: Added the 'Register' button on the login screen that navigates guests to the registration form.
- Admin Dashboard (adminDashboardControl + adminDashboard.fxml + adminDashBoard.css): Implemented the admin hub screen showing a personalized welcome Label and navigation buttons routing to Amenity Management, Room Type Management, and Room Management sub-modules. Implemented logout and exit functionality.

Abdallah Nageh (Guest Screens , Chatting)

- Guest Dashboard (GuestController + GuestDashBoard.fxml): Implemented the guest hub screen with dynamic welcome/balance Label and navigation buttons to all guest sub-screens (View Rooms, My Reservations, Chat, Logout).
- Room Browsing & Reservation (ViewAvailableRoomController + ViewAvailableRoom.fxml): Built the TreeView-based room browser that hierarchically displays room details (type, price, amenities). Integrated DatePicker check-in/out selection, wired to Guest.makeReservation(), and handled RoomNotAvailableException with descriptive Alert dialogs.
- My Reservations & Checkout (MyReservationControl + MyReservations.fxml): Built the full reservations table with color-coded row factory (PENDING / CONFIRMED / CHECKED_IN / COMPLETED / CANCELLED). Implemented cancel confirmation dialog and self-checkout flow connected to Guest.checkout(), catching InvalidPaymentException and refreshing balance after payment.
- Registration Screen (RegisterController + Register.fxml): Implemented all client-side validation: password policy check, DOB age restriction, balance non-negative validation, and password confirmation match. Integrated with Guest.register() for backend duplicate username detection.
- Application Bootstrap (MainClass): Set up the full startup sequence — seeding rooms, guests and reservations, launching the ChatServer in a background daemon thread, and showing the login screen.

Beshoy Hany (Room Type Management)

- Room Type Screen (RoomTypeController + RoomType.fxml + Roomtype.css): Implemented full CRUD for RoomType entities. Built the TableView with columns for ID, name, description, and base price. Used a ComboBox<RoomTypeName> to restrict type names to valid enum values. Validated price inputs and showed inline error Labels for invalid entries.
- Edit Logic: Implemented partial-update editing — the admin selects a row and fills only the fields they want to change; unchanged fields retain their current values.
- Delete with Cascade: On deletion of a RoomType, all Room objects in HotelDatabase sharing that type are also removed, maintaining data consistency.
- Back Navigation: Implemented back navigation from the Room Type screen to the Admin Dashboard.

Awsam Nady (Room Management)

- Room Management Screen (ManageRoomsController + RoomManagement.fxml + RoomManagement.css): Implemented the most complex admin screen. Built a live ComboBox<RoomType> that re-queries HotelDatabase on every open so newly added room types appear immediately — no restart needed.
- Multi-Select Amenities: Implemented a MenuButton with CheckMenuItems (one per Amenity in HotelDatabase) for multi-select amenity assignment. The button label updates dynamically to list all selected amenity names. Also refreshes on open for live consistency.
- Add Room: Validates room number uniqueness, positive price, and selected type before creating and adding the Room to HotelDatabase and the TableView.
- Edit Room: Supports partial updates — changes only the fields that the admin has filled in; other attributes remain unchanged. Prevents reassigning to a duplicate room number.
- Delete Room: Removes the selected room from HotelDatabase and refreshes the table. Shows a warning if no row is selected.

Hassan Abdallah (Receptionist Screens)

- Receptionist Hub (ReceptionistController + ManageReceptionistWindow.fxml): Built the sidebar navigation hub using a StackPane content area. Each sidebar button loads the corresponding sub-screen as a child Pane without replacing the main scene — keeping the sidebar persistent. Implemented logout that clears the current receptionist session and returns to login.
- Receptionist Dashboard (ReceptionistDashboardController + ManageReceptionistDashboard.fxml): Displays live summary statistics: total registered guests, number of available rooms (stream + filter), and count of active reservations (CONFIRMED + CHECKED_IN).
- Check-In / Check-Out (ReceptionistCheckInController + ManageReceptionistCheckIn.fxml): Implemented check-in via Receptionist.manageCheckIn(). For check-out, implemented a custom flow: finds the CHECKED_IN reservation, calculates nights and total, validates guest balance, deducts payment, creates an Invoice with the selected PaymentMethod, updates reservation to COMPLETED, and reveals a formatted invoice receipt VBox.
- Reservations View with Confirm (ReceptionistReservationsController + ManageReceptionistReservations.fxml): Built the full reservations table with five-status color-coded row factory. Implemented the Confirm button that calls Reservation.confirm() and prevents confirming non-PENDING reservations, with inline Label feedback.
- Chat Integration: Wired the sidebar Chat button to open the ChatWindow in a new Stage with the receptionist's session, allowing real-time communication with guests.

Testing & Verification

Valid Scenarios

Test ID	Action	Expected Result	Status
T-01	Login as Admin with correct credentials	Redirected to Admin Dashboard, welcome label shows username	PASS
T-02	Login as Receptionist (karim.r / Karim@456)	Receptionist hub loads with sidebar navigation	PASS
T-03	Login as Guest (ahmed / Ahmed123)	Guest Dashboard shows balance and username	PASS
T-04	Register new guest with valid data	Account created, redirected to login screen	PASS
T-05	Guest browses available rooms	TreeView shows all available rooms with amenities	PASS
T-06	Guest books an available room	Reservation created, room marked unavailable, success dialog shown	PASS
T-07	Guest views My Reservations	Table shows all reservations with correct color-coding by status	PASS
T-08	Guest cancels a PENDING reservation	Status changes to CANCELLED, room freed, table refreshes	PASS
T-09	Guest self-checkouts a CONFIRMED reservation with sufficient balance	Invoice dialog shown, balance deducted, reservation COMPLETED	PASS
T-10	Receptionist confirms a PENDING reservation	Status changes to CONFIRMED, success message shown inline	PASS
T-11	Receptionist checks in a guest	Reservation status → CHECKED_IN, success message shown	PASS
T-12	Receptionist checks out a guest	Invoice VBox appears with full receipt details, room freed	PASS
T-13	Admin adds a new room with amenities	Room appears in table, ComboBox shows DB room types	PASS
T-14	Admin edits room price	Table refreshes with new price immediately	PASS
T-15	Admin deletes amenity	Amenity removed from all rooms and from amenity table	PASS
T-16	Guest connects to chat and sends message to receptionist	Message appears in both guest and receptionist chat windows in real-time	PASS
T-17	Receptionist broadcasts to all guests	All connected guest chat windows receive the message instantly	PASS

Invalid / Edge Scenarios

Test ID	Action	Expected Result	Status
T-18	Login with wrong password	Error label displayed, no navigation	PASS
T-19	Register with password < 8 chars or no digit/uppercase	Inline error: password policy message shown	PASS
T-20	Register with existing username	Inline error: 'Username already exists'	PASS
T-21	Register with negative balance	Inline error: balance must be non-negative	PASS
T-22	Guest attempts checkout with insufficient balance	InvalidPaymentException caught, error dialog shown	PASS
T-23	Guest tries to cancel a COMPLETED reservation	IllegalStateException caught, error dialog shown	PASS
T-24	Receptionist confirms a non-PENDING reservation	Inline message: 'Only PENDING reservations can be confirmed'	PASS
T-25	Guest sends chat message without connecting first	Error dialog: 'Please connect to the server first'	PASS
T-26	Admin adds room with duplicate room number	Warning label: 'Room number already exists'	PASS
T-27	Admin adds room with non-numeric price	Warning label: 'Room Number and Price must be valid numbers'	PASS

Technical Specifications

Component	Technology / Version
Language	Java 17+
GUI Framework	JavaFX (FXML + CSS)
Build Tool	Maven (Spring Boot parent POM)
Networking	Java Sockets — ServerSocket / Socket (TCP, port 5555)
Threading	Java Thread (daemon), Platform.runLater() for FX-thread safety
IDE	IntelliJ IDEA (project .idea files included)
FXML Files	16 screens (all with corresponding controllers)
CSS Files	4 stylesheets (login, admin dashboard, amenity, room management)
Data Persistence	In-memory (HotelDatabase static ArrayLists), seeded by MainClass

Limitations

- No persistent storage: all data is held in memory and is lost when the application closes. Integration with a relational database (e.g., MySQL via JDBC or Hibernate) would resolve this.
- Chat requires manual connection: the guest or receptionist must click 'Connect' before sending messages. Auto-connect on window open would improve UX.
- Chat works on localhost only: the server address is hardcoded to 'localhost:5555'. A configuration file or settings screen would be needed for networked deployment.
- No password hashing: credentials are stored and compared as plain strings in HotelDatabase. A production system would require bcrypt or similar hashing.
- Chat history is not persisted: messages exist only in the TextArea for the duration of the session.

Conclusion

Milestone 2 successfully completes the Hotel Management System by building a fully functional JavaFX desktop application on top of the Milestone 1 backend. The system demonstrates a clean MVC-inspired architecture, effective use of multi-threading with Platform.runLater() for thread-safe UI updates, and a working real-time TCP chat system.

All five team members contributed meaningfully to distinct parts of the application — from the guest and receptionist workflows to the admin management screens and the chat networking layer. Every required feature from the project specification has been implemented and tested, including all mandatory JavaFX screens, FXML layouts, CSS styling, user feedback, and the bonus multi-threading and networking features.

The project represents a complete, runnable application that brings together the full spectrum of concepts taught in CSE241: object-oriented design, event-driven GUI programming, concurrent threads, and network socket communication.